



*Some parts of the notebook are almost the copy of [mmta-team course](#). Special thanks to [mmta-team](#) for making them publicly available. [Original notebook](#).*

Прочитайте семинар, пожалуйста, для успешного выполнения домашнего задания. В конце ноутбука напишите свой вывод. Работа без вывода оценивается ниже.

## Задача поиска схожих по смыслу предложений

Мы будем ранжировать вопросы [StackOverflow](#) на основе семантического векторного представления.

До этого в курсе не было речи про задачу ранжирования, поэтому введем её математическую формулировку.

## Задача ранжирования(Learning to Rank)

- $X$  - множество объектов
- $X^l = \{x_1, x_2, \dots, x_l\}$  - обучающая выборка На обучающей выборке задан порядок между некоторыми элементами, то есть нам известно, что некий объект выборки более релевантный для нас, чем другой:
- $i < j$  - порядок пары индексов объектов на выборке  $X^l$  с индексами  $i$  и  $j$

Задача:

построить ранжирующую функцию  $a : X \rightarrow R$  такую, что

$$i < j \Rightarrow a(x_i) < a(x_j)$$

## Embeddings

Будем использовать предобученные векторные представления слов на постах Stack Overflow. [A word2vec model trained on Stack Overflow posts](#)

Скачивание в colab

```
# !wget https://zenodo.org/record/1199620/files/SO_vectors_200.bin?download=1
```

Скачивание на локальный ПК

```
import os
import requests

URL = 'https://zenodo.org/record/1199620/files/SO_vectors_200.bin?download=1'
FOLDER = 'content'
PATH = os.path.join(FOLDER, 'SO_vectors_200.bin')

# Проверка наличия папки и создание по необходимости
os.makedirs(FOLDER, exist_ok=True)

# Проверка наличия файла и скачивание по необходимости
if not os.path.exists(PATH):
    r = requests.get(URL)
    with open(PATH, 'wb') as f:
        f.write(r.content)
    print('Файл скачан:', PATH)
else:
    print('Файл уже скачан')

Файл уже скачан

# !pip install gensim

from gensim.models.keyedvectors import KeyedVectors
# wv_embeddings =
KeyedVectors.load_word2vec_format("SO_vectors_200.bin?download=1",
binary=True)
wv_embeddings =
KeyedVectors.load_word2vec_format("content/SO_vectors_200.bin",
binary=True)
```

Как пользоваться этими векторами?

Рассмотрим на примере одного слова, что из себя представляет embedding.

```
word = 'dog'
if word in wv_embeddings:
    print(wv_embeddings[word].dtype, wv_embeddings[word].shape)

float32 (200,)

print(f"Num of words: {len(wv_embeddings.index_to_key)}")

Num of words: 1787145
```

Найдем наиболее близкие слова к слову `dog`:

### Вопрос 1:

- Входит ли слово `cat` в топ-5 близких слов к слову `dog`?
- Какое место занимает слово `cat` в наиболее близких словах к слову `dog`?

```
# method most_similar
'''your code'''

similar = wv_embeddings.most_similar('dog', topn=5)

print("Топ-5 слов близких слов к слову 'dog':")
for i, (w, s) in enumerate(similar, 1):
    print(f'{i}. {w}: {s:.4f}')

Топ-5 слов близких слов к слову 'dog':
1. animal: 0.8564
2. dogs: 0.7881
3. mammal: 0.7624
4. cats: 0.7621
5. animals: 0.7608

similar_full = wv_embeddings.most_similar('dog',
topn=len(wv_embeddings))

for i, (w, s) in enumerate(similar_full, 1):
    if w == 'cat':
        print(f'{i}. {w}: {s:.4f}')
        break

26. cat: 0.6852
```

**Ваш ответ:** слово `cat` не входит в топ-5 близких слов к слову `dog`;  
Слово `cat` только на 26-м месте среди ближайших слов к слову `dog`

## Векторные представления текста

Перейдем от векторных представлений отдельных слов к векторным представлениям вопросов, как к **среднему** векторов всех слов в вопросе. Если для какого-то слова нет предобученного вектора, то его нужно пропустить. Если вопрос не содержит ни одного известного слова, то нужно вернуть нулевой вектор.

```

import numpy as np
import re
# you can use your tokenizer
# for example, from nltk.tokenize import WordPunctTokenizer
class MyTokenizer:
    def __init__(self):
        pass
    def tokenize(self, text):
        return re.findall(r'\w+', text)
tokenizer = MyTokenizer()

def question_to_vec(question, embeddings, tokenizer, dim=200):
    """
        question: строка
        embeddings: наше векторное представление
        dim: размер любого вектора в нашем представлении

        return: векторное представление для вопроса
    """

    '''your code'''

    tokens = tokenizer.tokenize(question.lower())
    vec = np.zeros(dim, dtype='float32')
    cnt = 0
    for w in tokens:
        if w in embeddings:
            vec += embeddings[w]
            cnt += 1
    if cnt > 0:
        vec = vec / cnt
    return vec

```

Теперь у нас есть метод для создания векторного представления любого предложения.

### Вопрос 2:

- Какая третья(с индексом 2) компонента вектора предложения I love neural networks (округлите до 2 знаков после запятой)?

```

'''your code'''

# Предложение
question = 'I love neural networks'

vec = question_to_vec(question, wv_embeddings, tokenizer)
print(f"Третья (с индексом 2) компонента вектора предложения 'I love neural networks': {vec[2]:.2f}")

Третья (с индексом 2) компонента вектора предложения 'I love neural networks': -1.29

```

**Ваш ответ:** Третья (с индексом 2) компонента вектора предложения 'I love neural networks': -1.29

## Оценка близости текстов

Представим, что мы используем идеальные векторные представления слов. Тогда косинусное расстояние между дублирующими предложениями должно быть меньше, чем между случайно взятыми предложениями.

Сгенерируем для каждого из  $N$  вопросов  $R$  случайных отрицательных примеров и примешаем к ним также настоящие дубликаты. Для каждого вопроса будем ранжировать с помощью нашей модели  $R+1$  примеров и смотреть на позицию дубликата. Мы хотим, чтобы дубликат был первым в ранжированном списке.

### Hits@K

Первой простой метрикой будет количество корректных попаданий для какого-то  $K$ :

$$\text{Hits@K} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{rank}'_i \leq K],$$

- $\mathbb{I}[x < 0] \equiv \begin{cases} 1, & x < 0 \\ 0, & x \geq 0 \end{cases}$  - индикаторная функция
- $q_i$  -  $i$ -ый вопрос
- $q'_i$  - его дубликат
- $\text{rank}'_i$  - позиция дубликата в ранжированном списке ближайших предложений для вопроса  $q_i$ .

**Hits@K** измеряет долю вопросов, для которых правильный ответ попал в топ- $K$  позиций среди отранжированных кандидатов.

### DCG@K

Второй метрикой будет упрощенная DCG метрика, учитывающая порядок элементов в списке путем домножения релевантности элемента на вес равный обратному логарифму номера позиции:

$$\text{DCG@K} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\log_2(1 + \text{rank}'_i)} \cdot \mathbb{I}[\text{rank}'_i \leq K],$$

С такой метрикой модель штрафует за большой ранк корректного ответа.

**DCG@K** измеряет качество ранжирования, учитывая не только факт наличия правильного ответа в топ- $K$ , но и *его точную позицию*.

## Пример оценок

Вычислим описанные выше метрики для игрушечного примера. Пусть

- $N=1, R=3$
- "Что такое python?" - вопрос  $q_1$
- "Что такое язык python?" - его дубликат  $q'_i$

Пусть модель выдала следующий ранжированный список кандидатов:

1. "Как изучить с++?"
2. "Что такое язык python?"
3. "Хочу учить Java"
4. "Не понимаю Tensorflow"

$$\Rightarrow rank'_i = 2$$

Вычислим метрику  $DCG@K$  для  $K = 1, 4$ :

- $[K = 1] DCG@1 = \frac{1}{\log_2(1+2)} \cdot [2 \leq 1] = 0$
- $[K = 4] DCG@4 = \frac{1}{\log_2(1+2)} \cdot [2 \leq 4] = \frac{1}{\log_2 3}$

Вычислим метрику  $Hits@K$  для  $K = 1, 4$ :

- $[K = 1] Hits@1 = [rank'_i \leq 1]$

Проверяем условие  $\text{rank}_{q'_1} \leq 1$ : **условие неверно**.

Следовательно,  $[rank_{q'_1} \leq 1] = 0$ .

$$[K = 1] Hits@1 = 0$$

- $[K = 4] Hits@4 = [rank'_i \leq 4]$

Проверяем условие  $\text{rank}_{q'_1} \leq 4$ : **условие верно**.

$$[K = 4] Hits@4 = 1$$

Вычислим метрику  $DCG@K$  для  $K = 1, 4$ :

- $[K = 1] DCG@1 = \frac{1}{\log_2(1+2)} \cdot [2 \leq 1] = 0$
- $[K = 4] DCG@4 = \frac{1}{\log_2(1+2)} \cdot [2 \leq 4] = \frac{1}{\log_2 3}$

### Вопрос 3:

- Вычислите  $DCG@10$ , если  $rank'_i = 9$  (округлите до одного знака после запятой).

**Ваше решение:**

```
dcg = 1 / np.log2(10)
print(f'DCG@10 = {dcg:.1f}')
DCG@10 = 0.3
```

**Ваш ответ:** DCG@10 = 0.3

Более сложный пример оценок

Рассмотрим пример с  $N > 1$ , где  $N = 3$  (три вопроса) и для каждого вопроса заданы позиции их дубликатов. Вычислим метрики Hits@K для разных значений K.

---

- $N = 3$ : Три вопроса  $q_1, q_2, q_3$ .
- Для каждого вопроса известна позиция его дубликата ( $\text{rank}_{q'_i}$ ):
  - $\text{rank}_{q'_1} = 2$ ,
  - $\text{rank}_{q'_2} = 5$ ,
  - $\text{rank}_{q'_3} = 1$ .

Мы будем вычислять Hits@K для  $K = 1, 5$ .

---

**Для  $K = 1$ :**

Подставим значения:

$$\text{Hits}@1 = \frac{1}{3} \cdot \left( \left[ \text{rank}_{q'_1} \leq 1 \right] + \left[ \text{rank}_{q'_2} \leq 1 \right] + \left[ \text{rank}_{q'_3} \leq 1 \right] \right).$$

Проверяем условие  $\text{rank}_{q'_i} \leq 1$  для каждого вопроса:

- $\text{rank}_{q'_1} = 2 \rightarrow 2 \not\leq 1 \rightarrow 0$ ,
- $\text{rank}_{q'_2} = 5 \rightarrow 5 \not\leq 1 \rightarrow 0$ ,
- $\text{rank}_{q'_3} = 1 \rightarrow 1 \leq 1 \rightarrow 1$ .

Сумма:

$$\text{Hits}@1 = \frac{1}{3} \cdot (0 + 0 + 1) = \frac{1}{3}.$$

$$\text{Hits}@1 = \frac{1}{3}.$$

---

**Для  $K = 5$ :**

Подставим значения:

$$\text{Hits}@5 = \frac{1}{3} \cdot \left( [\text{rank}_{q'_1} \leq 5] + [\text{rank}_{q'_2} \leq 5] + [\text{rank}_{q'_3} \leq 5] \right).$$

Проверяем условие  $\text{rank}_{q'_i} \leq 5$  для каждого вопроса:

- $\text{rank}_{q'_1} = 2 \rightarrow 2 \leq 5 \rightarrow 1$ ,
- $\text{rank}_{q'_2} = 5 \rightarrow 5 \leq 5 \rightarrow 1$ ,
- $\text{rank}_{q'_3} = 1 \rightarrow 1 \leq 5 \rightarrow 1$ .

Сумма:

$$\text{Hits}@5 = \frac{1}{3} \cdot (1+1+1) = 1.$$

$$\text{Hits}@5 = 1.$$

Теперь вычислим метрику **DCG@K** для того же примера, где  $N = 3$  (три вопроса), и для каждого вопроса известна позиция его дубликата ( $\text{rank}_{q'_i}$ ):

- $\text{rank}_{q'_1} = 2$ ,
- $\text{rank}_{q'_2} = 5$ ,
- $\text{rank}_{q'_3} = 1$ .

Мы будем вычислять **DCG@K** для  $K = 1, 5$ .

---

**Для  $K = 1$ :** Подставим значения:

$$\text{DCG}@1 = \frac{1}{3} \cdot \left( \frac{1}{\log_2(1 + \text{rank}_{q'_1})} \cdot [\text{rank}_{q'_1} \leq 1] + \frac{1}{\log_2(1 + \text{rank}_{q'_2})} \cdot [\text{rank}_{q'_2} \leq 1] + \frac{1}{\log_2(1 + \text{rank}_{q'_3})} \cdot [\text{rank}_{q'_3} \leq 1] \right).$$

Проверяем условие  $\text{rank}_{q'_i} \leq 1$  для каждого вопроса:

- $\text{rank}_{q'_1} = 2 \rightarrow 2 \not\leq 1 \rightarrow 0$ ,
- $\text{rank}_{q'_2} = 5 \rightarrow 5 \not\leq 1 \rightarrow 0$ ,
- $\text{rank}_{q'_3} = 1 \rightarrow 1 \leq 1 \rightarrow 1$ .

Сумма:

$$\text{DCG}@1 = \frac{1}{3} \cdot (0+0+1) = \frac{1}{3}.$$

$$\text{DCG}@1 = \frac{1}{3}.$$


---

**Для  $K = 5$ :** Подставим значения:

$$DCG@5 = \frac{1}{3} \cdot \left( \frac{1}{\log_2(1 + \text{rank}_{q'_1})} \cdot [\text{rank}_{q'_1} \leq 5] + \frac{1}{\log_2(1 + \text{rank}_{q'_2})} \cdot [\text{rank}_{q'_2} \leq 5] + \frac{1}{\log_2(1 + \text{rank}_{q'_3})} \cdot [\text{rank}_{q'_3} \leq 5] \right).$$

Проверяем условие  $\text{rank}_{q'_i} \leq 5$  для каждого вопроса:

- $\text{rank}_{q'_1} = 2 \rightarrow 2 \leq 5 \rightarrow 1$ ,
- $\text{rank}_{q'_2} = 5 \rightarrow 5 \leq 5 \rightarrow 1$ ,
- $\text{rank}_{q'_3} = 1 \rightarrow 1 \leq 5 \rightarrow 1$ .

Сумма:

$$DCG@5 = \frac{1}{3} \cdot (0.631 + 0.387 + 1) = \frac{1}{3} \cdot 2.018 \approx 0.673.$$

$$DCG@5 \approx 0.673.$$

#### Вопрос 4:

- Какое максимальное значение может принимать Hits@47 - DCG@1?

#### Ваше решение:

Условие:

Hits@47 = 1 при rank от 1 до 47 (иначе 0).

DCG@1 = 1 только при rank равным 1 (иначе 0).

Вариант при rank = 1:

$$\text{Hits@47} - \text{DCG@1} = [1 \leq 47] - [1 == 1] = 1 - 1 = 0$$

Вариант при rank >= 48:

$$\text{Hits@47} - \text{DCG@1} = [48 \leq 47] - [48 == 1] = 0 - 0 = 0$$

Максимум будет достигаться при rank от 2 до 47 (1 - 0) и будет равен 1:

$$\text{Hits@47} - \text{DCG@1} = [2 \leq 47] - [2 == 1] = 1 - 0 = 1$$

**Ваш ответ:** Hits@47 - DCG@1 может принимать максимальное значение равное 1

## HITS\_COUNT и DCG\_SCORE

Каждая функция имеет два аргумента: *dup\_ranks* и *k*.

*dup\_ranks* является списком, который содержит рейтинги дубликатов (их позиции в ранжированном списке).

К примеру для "Что такое язык python?"  $\text{dup_ranks} = [2]$ .

```
def hits_count(dup_ranks, k):
    """
        dup_ranks: list индексов дубликатов
        k: пороговое значение для ранга
        result: вернуть Hits@k
    """
```

```

'''your code'''

if not dup_ranks:
    return 0

# Подсчитываем количество дубликатов, чей ранг <= k
total = 0
for r in dup_ranks:
    if r <= k:
        total += 1

# Делим на общее количество вопросов
hits_value = total / len(dup_ranks)

return hits_value

# Расчитаем метрику Hits с помощью реализованных функций
dup_ranks = [2]

hits_value = hits_count(dup_ranks, k=1)
print(f"Hits@1 = {hits_value}")

hits_value = hits_count(dup_ranks, k=4)
print(f"Hits@4 = {hits_value}")

Hits@1 = 0.0
Hits@4 = 1.0

def dcg_score(dup_ranks, k):
    """
        dup_ranks: list индексов дубликатов
        k: пороговое значение для ранга
        result: вернуть DCG@k
    """
    '''your code'''

    if not dup_ranks:
        return 0

    # Вычисляем сумму для всех релевантных дубликатов
    total = 0.0
    for r in dup_ranks:
        if r <= k:
            total += 1.0 / np.log2(1 + r)

    # Делим на общее количество вопросов
    dcg_value = total / len(dup_ranks)

    return dcg_value

```

```
# Рассчитаем метрику DCG с помощью реализованных функций
dup_ranks = [2]
```

```
# Вычисляем DCG@1
dcg_value = dcg_score(dup_ranks, k=1)
print(f"DCG@1 = {dcg_value:.3f}")
```

```
# Вычисляем DCG@4
dcg_value = dcg_score(dup_ranks, k=4)
print(f"DCG@4 = {dcg_value:.3f}")
```

DCG@1 = 0.000

DCG@4 = 0.631

Протестируем функции. Пусть  $N=1$ , то есть у нас один эксперимент. Будем искать копию вопроса и оценивать метрики.

```
import pandas as pd
```

```
copy_answers = ["How does the catch keyword determine the type of  
exception that was thrown",]
```

## # наши кандидаты

[illegible]

```
# dup_ranks — позиции наших копий, так как эксперимент один, то этот массив длины 1
```

```
dup_ranks = [2]
```

# вычисляем метрику для разных  $k$

```
print('Ваш ответ HIT:', [hits_count(dup_ranks, k) for k in range(1, 5)])
```

```
print('Ваш ответ DCG:', [round(dcg_score(dup_ranks, k), 5) for k in
range(1, 5)])
```

Ваш ответ HIT: [0.0, 1.0, 1.0, 1.0]

Ваш ответ DCG: [0.0, 0.63093, 0.63093, 0.63093]

У вас должно получиться:

# correct\_answers - метрика для разных k

```
correct_answers = pd.DataFrame([[0, 1, 1, 1], [0, 1 / (np.log2(3)), 1 / (np.log2(3)), 1 / (np.log2(3))]],
                                index=['HITS', 'DCG'],
```

```
columns=range(1,5))
```

correct answers

|      | 1 | 2       | 3       | 4       |
|------|---|---------|---------|---------|
| HITS | 0 | 1.00000 | 1.00000 | 1.00000 |
| DCG  | 0 | 0.63093 | 0.63093 | 0.63093 |

*# Проверка*

```
assert(correct_answers.iloc[0].tolist()==[hits_count(dup_ranks, k) for
k in range(1, 5)])
assert(correct_answers.iloc[1].round(5).tolist()==[round(dcg_score(dup
_ranks, k), 5) for k in range(1, 5)])
```

## Данные

[arxiv link](#)

`train.tsv` - выборка для обучения. В каждой строке через табуляцию записаны: ,

`validation.tsv` - тестовая выборка. В каждой строке через табуляцию записаны: ,,  
**<отрицательный пример 1>**, **<отрицательный пример 2>**, ...

```
# !unzip stackoverflow_similar_questions.zip
```

Скачивание и распаковка на локальный ПК

```
import gdown
import zipfile

# Скачивание

FILE_ID = '1QqT4D0EoqJTy7v9VrNCYD-m964XZFR7_'
FOLDER = 'content'
PATH = os.path.join(FOLDER, 'stackoverflow_similar_questions.zip')

# Проверка наличия файла и скачивание по необходимости
if not os.path.exists(PATH):
    gdown.download(f'https://drive.google.com/uc?id={FILE_ID}', PATH,
quiet=True)
    print('Файл скачан:', PATH)
else:
    print('Файл уже скачан')

Файл уже скачан

# Распаковка

EXTRACT_FOLDER = 'content/stackoverflow_similar_questions'

if not os.path.exists(EXTRACT_FOLDER):
    with zipfile.ZipFile(PATH, 'r') as zf:
        zf.extractall(path=EXTRACT_FOLDER)
```

```
print('Файл распакован в:', EXTRACT_FOLDER)
else:
    print('Файл уже распакован')
```

Файл уже распакован

Считайте данные.

```
def read_corpus(filename):
    data = []
    with open(filename, encoding='utf-8') as file:
        for line in file:
            data.append(line.strip().split('\t'))
    return data
```

Нам понадобится только файл validation.

```
# validation_data = read_corpus('./data/validation.tsv')
validation_data = read_corpus(f'{EXTRACT_FOLDER}/data/validation.tsv')
```

Кол-во строк

```
len(validation_data)
```

3760

Размер нескольких первых строк

```
for i in range(5):
    print(i + 1, len(validation_data[i]))

1 1001
2 1001
3 1001
4 1001
5 1001
```

## Ранжирование без обучения

Реализуйте функцию ранжирования кандидатов на основе косинусного расстояния. Функция должна по списку кандидатов вернуть отсортированный список пар (позиция в исходном списке кандидатов, кандидат). При этом позиция кандидата в полученном списке является его рейтингом (первый - лучший). Например, если исходный список кандидатов был [a, b, c], и самый похожий на исходный вопрос среди них - c, затем a, и в конце b, то функция должна вернуть список [(2, c), (0, a), (1, b)].

```
from sklearn.metrics.pairwise import cosine_similarity
from copy import deepcopy
```

```

def rank_candidates(question, candidates, embeddings, tokenizer,
dim=200):
    """
        question: строка
        candidates: массив строк(кандидатов) [a, b, c]
        result: пары (начальная позиция, кандидат) [(2, c), (0, a),
(1, b)]
    """
    '''your code'''

    q_vec = question_to_vec(question, embeddings, tokenizer, dim)
    scores = []
    for idx, cand in enumerate(candidates):
        c_vec = question_to_vec(cand, embeddings, tokenizer, dim)
        sim = cosine_similarity([q_vec], [c_vec])[0, 0]
        scores.append((idx, cand, sim))
    scores.sort(key=lambda x: x[2], reverse=True)
    result = [(idx, cand) for idx, cand, _ in scores]

    return result

```

Протестируйте работу функции на примерах ниже. Пусть  $N=2$ , то есть два эксперимента

```

questions = ['converting string to list', 'Sending array via Ajax
fails']

candidates = [['Convert Google results object (pure js) to Python
object', # первый эксперимент
              'C# create cookie from string and send it',
              'How to use jQuery AJAX for an outside domain?'],
              ['Getting all list items of an unordered list in PHP',
# второй эксперимент
              'WPF- How to update the changes in list item of a
list',
              'select2 not displaying search results']]

for question, q_candidates in zip(questions, candidates):
    ranks = rank_candidates(question, q_candidates, wv_embeddings,
tokenizer)
    display(ranks)
    print()

[(1, 'C# create cookie from string and send it'),
 (0, 'Convert Google results object (pure js) to Python object'),
 (2, 'How to use jQuery AJAX for an outside domain?')]

```

```
[(0, 'Getting all list items of an unordered list in PHP'),  
 (2, 'select2 not displaying search results'),  
 (1, 'WPF- How to update the changes in list item of a list')]
```

Для первого эксперимента вы можете полностью сравнить ваши ответы и правильные ответы. Но для второго эксперимента два ответа на кандидаты будут скрыты(\*)

```
# ДОЛЖНО ВЫВЕСТИ  
results = [[(1, 'C# create cookie from string and send it'),  
             (0, 'Convert Google results object (pure js) to Python  
object'),  
             (2, 'How to use jQuery AJAX for an outside domain?')],  
            [( '*', 'Getting all list items of an unordered list in  
PHP'), #скрыт  
             ( '*', 'select2 not displaying search results'), #скрыт  
             ( '*', 'WPF- How to update the changes in list item of a  
list')]] #скрыт
```

Вы должны получить для эксперимента 1 следующую последовательность: 1, 0, 2.

#### **Вопрос 5:**

- Какую последовательность начальных индексов вы получили для эксперимента 2? (перечислите без запятой и пробелов, например, 102 для первого эксперимента)

**Ваш ответ:** 021

Теперь мы можем оценить качество нашего метода. Запустите следующие два блока кода для получения результата. Обратите внимание, что вычисление расстояния между векторами занимает некоторое время (примерно 10 минут). Можете взять для validation 1000 примеров.

```
# from tqdm.notebook import tqdm  
from tqdm.auto import tqdm  
  
wv_ranking = []  
max_validation_examples = 1000  
for i, line in enumerate(tqdm(validation_data)):  
    if i == max_validation_examples:  
        break  
    q, *ex = line  
    ranks = rank_candidates(q, ex, wv_embeddings, tokenizer)  
    wv_ranking.append([r[0] for r in ranks].index(0) + 1)  
  
{ "model_id": "93875c939a804e91aa77f7c365d5f0d6", "version_major": 2, "version_minor": 0 }
```

```
for k in tqdm([1, 5, 10, 100, 500, 1000]):
    print("DCG@%4d: %.3f | Hits@%4d: %.3f" % (k, dcg_score(wv_ranking,
k), k, hits_count(wv_ranking, k)))
```

```
{"model_id": "310189e4cbfe4385a5936f6671c82ed9", "version_major": 2, "version_minor": 0}
```

```
DCG@ 1: 0.415 | Hits@ 1: 0.415
DCG@ 5: 0.502 | Hits@ 5: 0.582
DCG@ 10: 0.525 | Hits@ 10: 0.651
DCG@ 100: 0.570 | Hits@ 100: 0.874
DCG@ 500: 0.583 | Hits@ 500: 0.973
DCG@1000: 0.586 | Hits@1000: 1.000
```

Из формул выше можно понять, что

- $\text{Hits@K}$  — **монотонно неубывающая функция**  $K$ , которая стремится к 1 при  $K \rightarrow \infty$ .
- $\text{DCG@K}$  — **монотонно неубывающая функция**  $K$ , но рост замедляется с увеличением  $K$  из-за убывания веса  $\frac{1}{\log_2(1 + \text{rank}_{(q'_i)})}$ .

## Эмбединги, обученные на корпусе похожих вопросов

```
# train_data = read_corpus('./data/train.tsv')
train_data = read_corpus(f'{EXTRACT_FOLDER}/data/train.tsv')
```

Улучшите качество модели. Склеим вопросы в пары и обучим на них модель Word2Vec из gensim. Выберите размер window. Объясните свой выбор.

*Рассмотрим подробнее* данное склеивание.

1. Каждая строка из train\_data разбивается на вопрос (question) и список кандидатов.
2. Для каждого кандидата вопрос склеивается с ним в одну строку.
3. Склеенная строка (combined\_text) токенизируется, и полученный список токенов добавляется в общий корпус (corpus).

### Пример

Вопрос: "What is Python?"

Кандидаты: ["Python is a programming language", "Java is another language"]

Склеенные строки:

"What is Python? Python is a programming language"

"What is Python? Java is another language"

Токенизированные списки:

```
['what', 'is', 'python', 'python', 'is', 'a', 'programming',  
'language']  
['what', 'is', 'python', 'java', 'is', 'another', 'language']
```

```
'''your code'''
```

*# Создаем общий корпус текстов*

```
def create_corpus(data, tokenizer):  
    words = []  
    lengths = []  
  
    for row in data:  
        question, positive = row[0], row[1]  
        combined = question + " " + positive  
        tokens = tokenizer.tokenize(combined.lower())  
        words.append(tokens)  
        lengths.append(len(tokens))  
  
    return words, lengths
```

```
words, lengths = create_corpus(train_data, tokenizer)
```

```
print(f'''Минимальная длина: {min(lengths)},  
Максимальная длина: {max(lengths)},  
Средняя длина: {sum(lengths) // len(lengths)}''')
```

```
Минимальная длина: 3,  
Максимальная длина: 63,  
Средняя длина: 17
```

```
from gensim.models import Word2Vec
```

```
embeddings_trained = Word2Vec(  
    sentences=words,          # Корпус токенизированных текстов  
    vector_size=200,         # Размерность векторов  
    window=5,               # Размер окна контекста (размер окна 5  
достаточно для улавливания контекста в вопросах средней длины)  
    min_count=5,            # Минимальная частота слов  
    seed=42,                # Фиксация случайности  
    workers=4               # Количество потоков  
)
```

```
def ranking(data, embeddings, tokenizer):  
    wv_ranking = []  
    max_validation_examples = 1000  
  
    for i, line in enumerate(tqdm(data)):  
        if i == max_validation_examples:  
            break
```

```

q, *ex = line
ranks = rank_candidates(q, ex, embeddings, tokenizer)
wv_ranking.append([r[0] for r in ranks].index(0) + 1)

return wv_ranking

wv_ranking = ranking(validation_data, embeddings_trained, tokenizer)

{"model_id": "9c701e0c9d3f4ea9b417c21468fa58db", "version_major": 2, "version_minor": 0}

for k in tqdm([1, 5, 10, 100, 500, 1000]):
    print("DCG@%4d: %.3f | Hits@%4d: %.3f" % (k, dcg_score(wv_ranking, k), k, hits_count(wv_ranking, k)))

{"model_id": "121c30dc636a4b33987759bea97d63c4", "version_major": 2, "version_minor": 0}

DCG@ 1: 0.308 | Hits@ 1: 0.308
DCG@ 5: 0.385 | Hits@ 5: 0.452
DCG@ 10: 0.412 | Hits@ 10: 0.535
DCG@ 100: 0.463 | Hits@ 100: 0.786
DCG@ 500: 0.484 | Hits@ 500: 0.944
DCG@1000: 0.490 | Hits@1000: 1.000

```

## Замечание:

Решить эту задачу с помощью обучения полноценной нейронной сети будет вам предложено, как часть задания в одной из домашних работ по теме "Диалоговые системы".

## Бонус

Попробуем альтернативные токенизаторы

```

# nltk

from nltk.tokenize import WordPunctTokenizer

class NLTKTokenizer:
    def __init__(self):
        self.tokenizer = WordPunctTokenizer()
    def tokenize(self, text):
        return self.tokenizer.tokenize(text)

tokenizer_nltk = NLTKTokenizer()

words_nltk, lengths_nltk = create_corpus(train_data, tokenizer_nltk)

embeddings_trained_nltk = Word2Vec(
    sentences=words_nltk, # Корпус токенизированных текстов

```

```

        vector_size=200,          # Размерность векторов
        window=5,                 # Размер окна контекста (размер окна 5
достаточно для улавливания контекста в вопросах средней длины)
        min_count=5,              # Минимальная частота слов
        seed=42,                  # Фиксация случайности
        workers=4                 # Количество потоков
    ).wv

wv_ranking_nltk = ranking(validation_data, embeddings_trained_nltk,
tokenizer_nltk)

{"model_id": "a20eaa2bd7ed49038b6ab4ea3dabc213", "version_major": 2, "version_minor": 0}

for k in tqdm([1, 5, 10, 100, 500, 1000]):
    print("DCG@%4d: %.3f | Hits@%4d: %.3f" % (k,
dcg_score(wv_ranking_nltk, k), k, hits_count(wv_ranking_nltk, k)))

{"model_id": "5316460a978d4c65b3ef1983b5b78e17", "version_major": 2, "version_minor": 0}

DCG@   1: 0.290 | Hits@   1: 0.290
DCG@   5: 0.369 | Hits@   5: 0.438
DCG@  10: 0.390 | Hits@  10: 0.506
DCG@ 100: 0.446 | Hits@ 100: 0.778
DCG@ 500: 0.467 | Hits@ 500: 0.944
DCG@1000: 0.473 | Hits@1000: 1.000

# import nltk
# nltk.download('stopwords')
# !python -m spacy download en_core_web_sm

from nltk.corpus import stopwords

stopWords = set(stopwords.words('english'))

# spacy со стоп-словами

import spacy

class SpacyTokenizer:
    def __init__(self):
        self.nlp = spacy.load('en_core_web_sm', disable=['parser',
'ner'])
    def tokenize(self, text):
        spacy_results = self.nlp(text)
        return [token.lemma_ for token in spacy_results if
token.lemma_ not in stopWords and not token.is_space]

tokenizer_spacy = SpacyTokenizer()

```

*# Долгое обучение...*

```
words_spacy, lenglhs_spacy = create_corpus(train_data, tokenizer_spacy)
```

```
embeddings_trained_spacy = Word2Vec(  
    sentences=words_spacy, # Корпус токенизированных текстов  
    vector_size=200,      # Размерность векторов  
    window=5,            # Размер окна контекста (размер окна 5  
    достаточно для улавливания контекста в вопросах средней длины)  
    min_count=5,          # Минимальная частота слов  
    seed=42,              # Фиксация случайности  
    workers=4             # Количество потоков  
) .wv
```

```
wv_ranking_spacy = ranking(validation_data, embeddings_trained_spacy,  
tokenizer_spacy)
```

```
{"model_id": "6508ffef45984df384be5b24f05f368c", "version_major": 2, "version_minor": 0}
```

```
for k in tqdm([1, 5, 10, 100, 500, 1000]):  
    print("DCG@%4d: %.3f | Hits@%4d: %.3f" % (k,  
dcg_score(wv_ranking_spacy, k), k, hits_count(wv_ranking_spacy, k)))
```

```
{"model_id": "e40e56a770da446ea03a0259768ffd70", "version_major": 2, "version_minor": 0}
```

|           |       |  |            |       |
|-----------|-------|--|------------|-------|
| DCG@ 1:   | 0.398 |  | Hits@ 1:   | 0.398 |
| DCG@ 5:   | 0.484 |  | Hits@ 5:   | 0.560 |
| DCG@ 10:  | 0.509 |  | Hits@ 10:  | 0.638 |
| DCG@ 100: | 0.554 |  | Hits@ 100: | 0.862 |
| DCG@ 500: | 0.568 |  | Hits@ 500: | 0.971 |
| DCG@1000: | 0.571 |  | Hits@1000: | 1.000 |

Напишите свой вывод о полученных результатах.

- Какой принцип токенизации даёт качество лучше и почему?
- Помогает ли нормализация слов?
- Какие эмбединги лучше справляются с задачей и почему?
- Почему получилось плохое качество решения задачи?
- Предложите свой подход к решению задачи.

## Вывод:

***Ваш вывод:***

- Из протестированных токенизаторов (с использованием регулярных выражений, nltk, spacy) для обученных с нуля эмбедингов токенизация с помощью spacy показала лучший результат, потому что используется лематизация и удаление стоп-слов.

Токенизация с использованием библиотеки nltk показала худший результат, так как не использовалась лематизация и нормализация слов.

- Нормализация слов улучшает обобщение. Но не всегда это полезно, так как иногда может исказиться смысл слов при нормализации.
- Предобученные эмбединги (SO\_vectors) дали лучшие метрики по сравнению с обученными на парах «вопрос-похожий вопрос», так как предобученная модель обучена на гораздо большем объеме данных. К примеру, Hits@1 равная 0.415 предобученной модели против ~0.300-320 обученной на ограниченном корпусе текста модели.
- Плохое качество может быть связано с потерей порядка слов (смысл предложений может меняться на противоположный) и игнорированием важности отдельных токенов из-за усреднения.
- Предложения по улучшению: использовать взвешенное усреднение (TF-IDF веса) для выделения значимых слов, удаление стоп-слов, увеличение размера вектора эмбедингов.